Computer Vision • Web Dashboard • Live Demo

Parking Detection Software

A prototype that detects parking spot status from camera input and publishes live availability to a reservation dashboard.

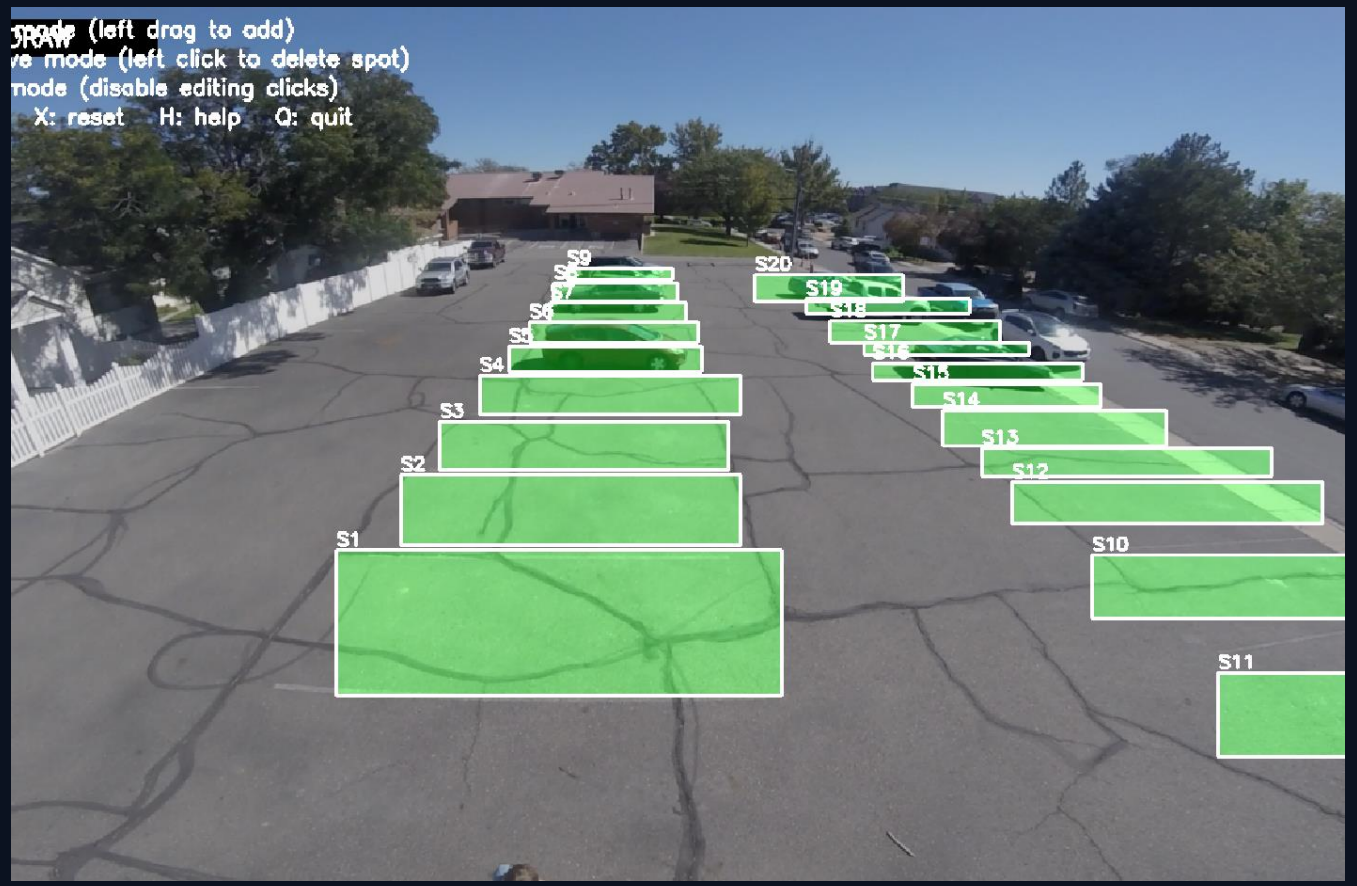
Student Showcase

github.com/DawsonHWright/ParkingDetectionSoftware

Purpose: make parking availability visible

The problem

Drivers cannot easily know which lots or spaces are available until they physically drive through them.



Spot masks define the parking spaces

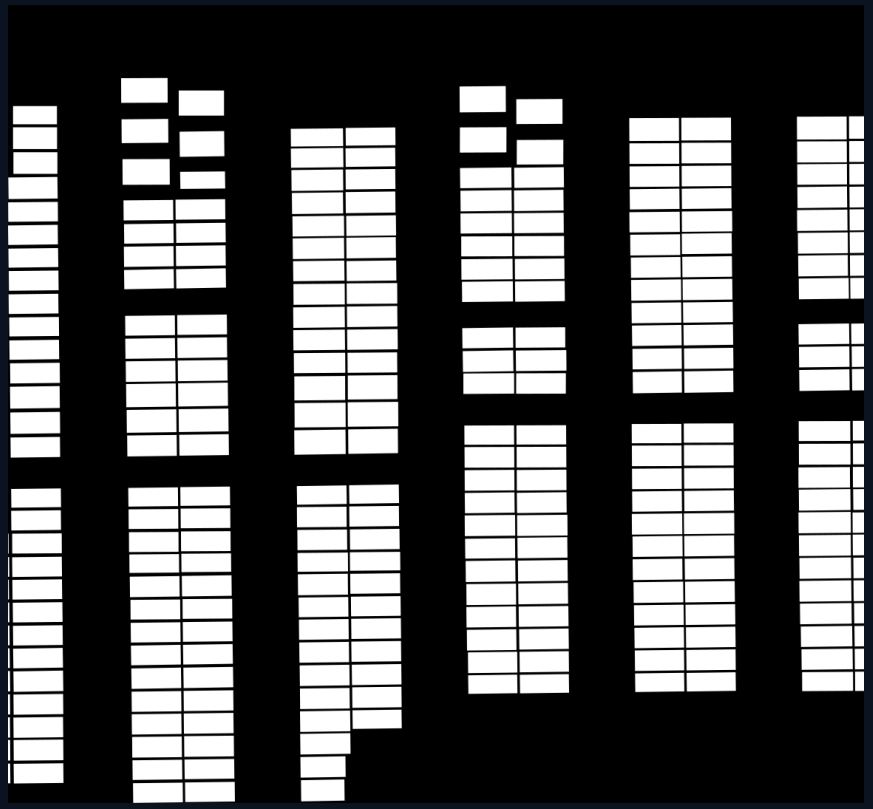
Detector turns vision into status

What the prototype proves

Camera or image input can be mapped into individual parking spots, classified as free or occupied, and passed to a dashboard that supports reservations.

Tools used across the prototype

Python	Runs the computer vision pipeline and command-line tools.
OpenCV + NumPy	Processes frames, reads and creates masks, computes edges, motion differences, and overlays status.
Flask	Provides backend API routes for status, reservation, and updates.
HTML / CSS / JavaScript	Builds the browser dashboard and user interaction layer.
JSON	Acts as the data format between detector, backend, and frontend.
Render	Hosts the backend API and runs the detector as a background worker.
GitHub	Version control and shared repository for code, docs, and demo assets.



Repository language mix

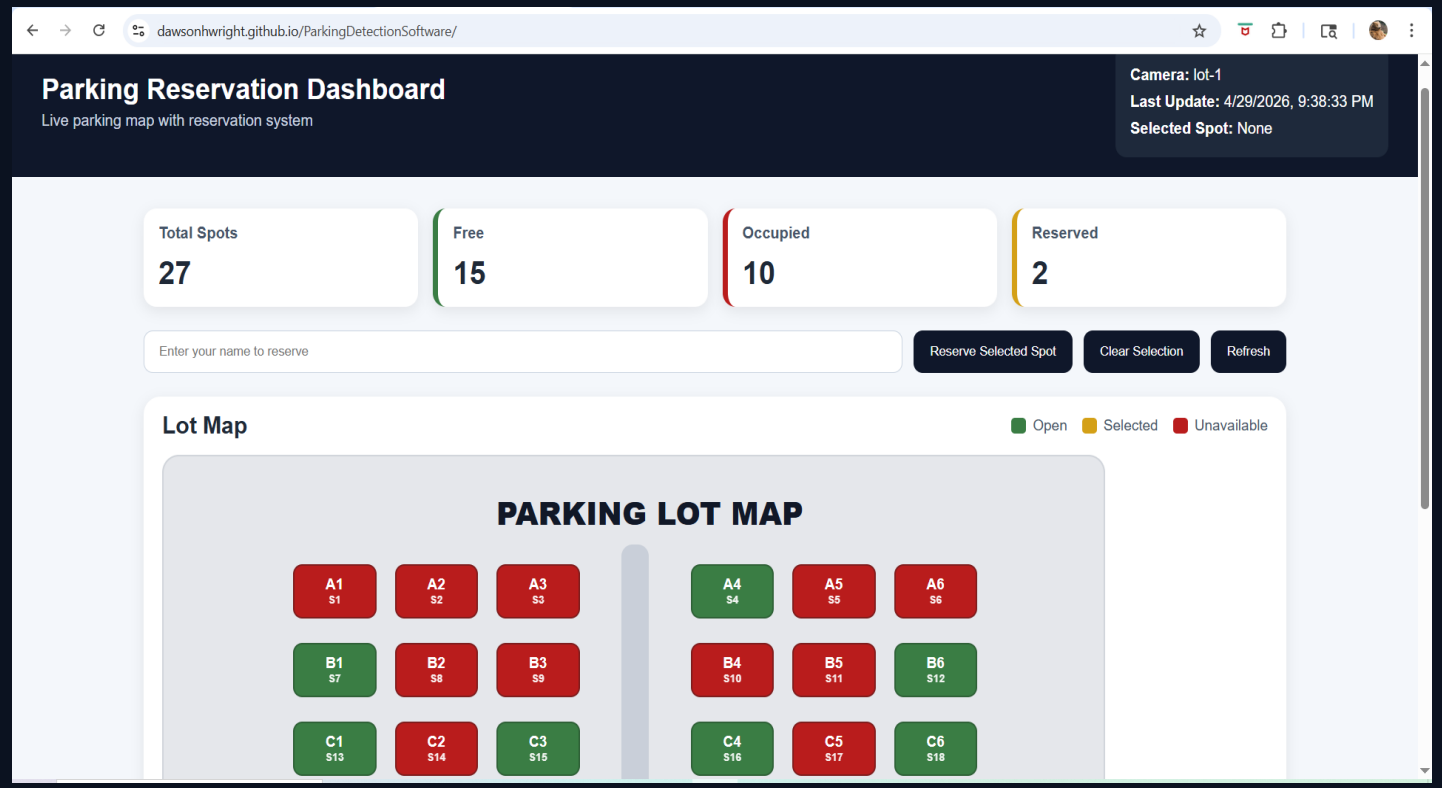
Python • JavaScript • CSS • HTML

How the system flows

Core idea

The computer vision layer does the sensing work, while the web layer turns that sensing into information people can actually use.

Dashboard layer: live status + reservations



What users see

A clean dashboard with total, free, occupied, and reserved counts, plus a spot-by-spot view.

What the server handles

Flask API routes handle status updates, reservations, and data syncing (hosted on Render).

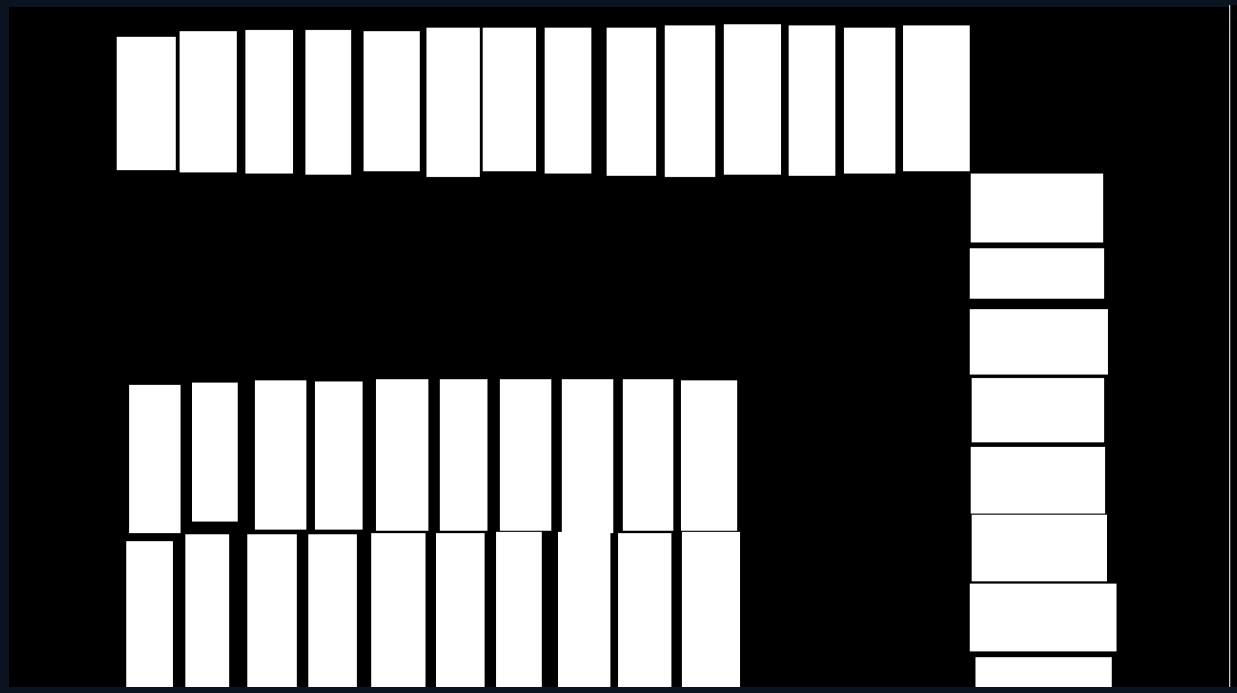
Why this matters

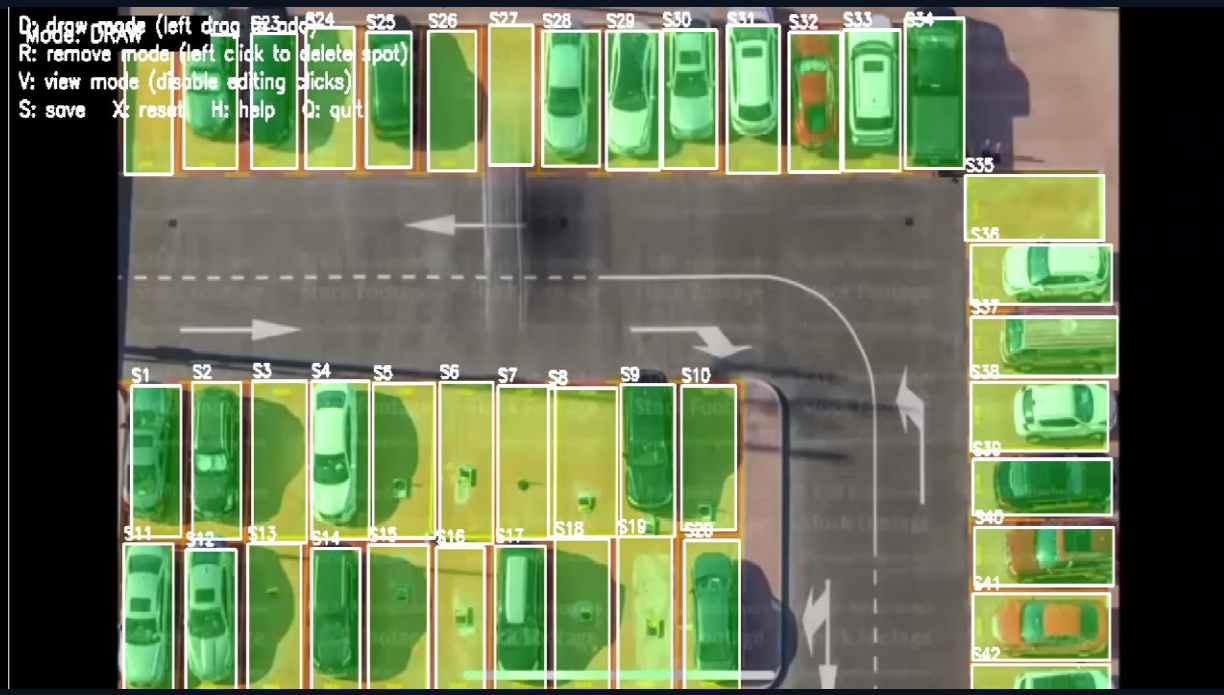
The project is not only detection. It shows how detection data can support an actual user workflow.

Ethical Concerns

Privacy concerns from camera input (people, license plates). Detection errors may impact user trust and fairness. Reservation misuse (holding spots without using them)

Computer vision layer: masks + detection





Mask Generator

Creates editable parking-space regions so each spot has a known area of interest.

Detector

Reads the frame and mask edge density is 75% and brightness changes is 25% , calculates visual change, then outputs free or occupied status for every spot. (Tests using videos and pictures of parking lots)

Live demo checkpoint

Demo flow

1. **Open the dashboard**

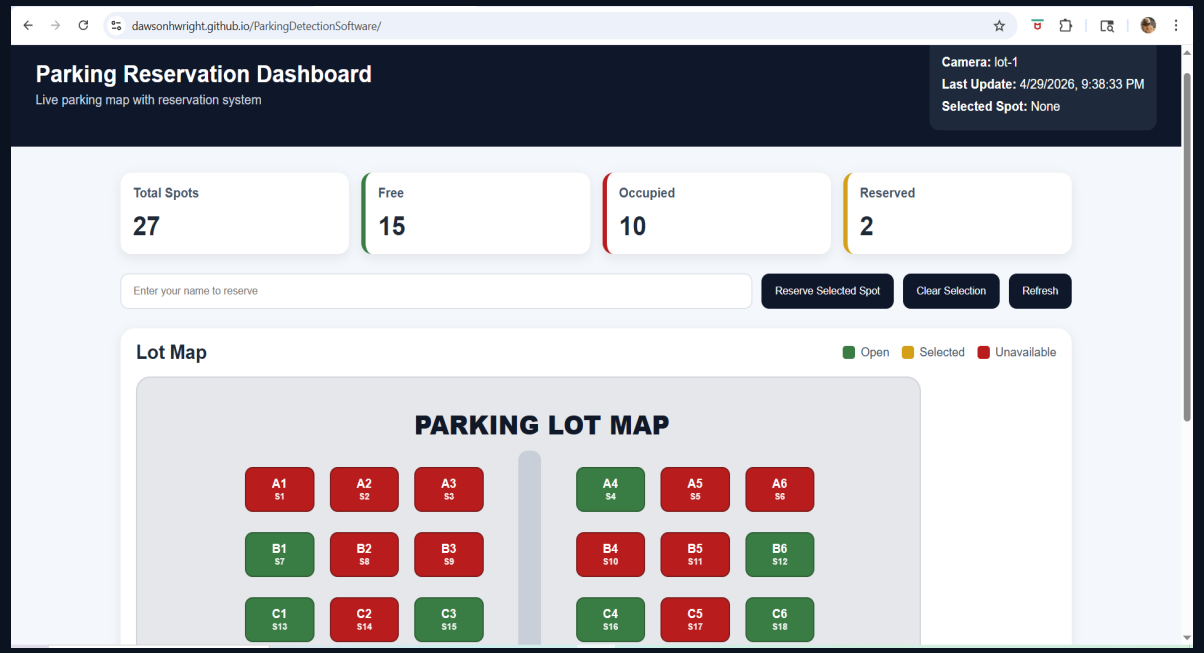
   Show the summary cards and current spot statuses.

2. **Run or explain detector input**

   Point out how camera/image data becomes status data.

3. **Reserve a free spot**

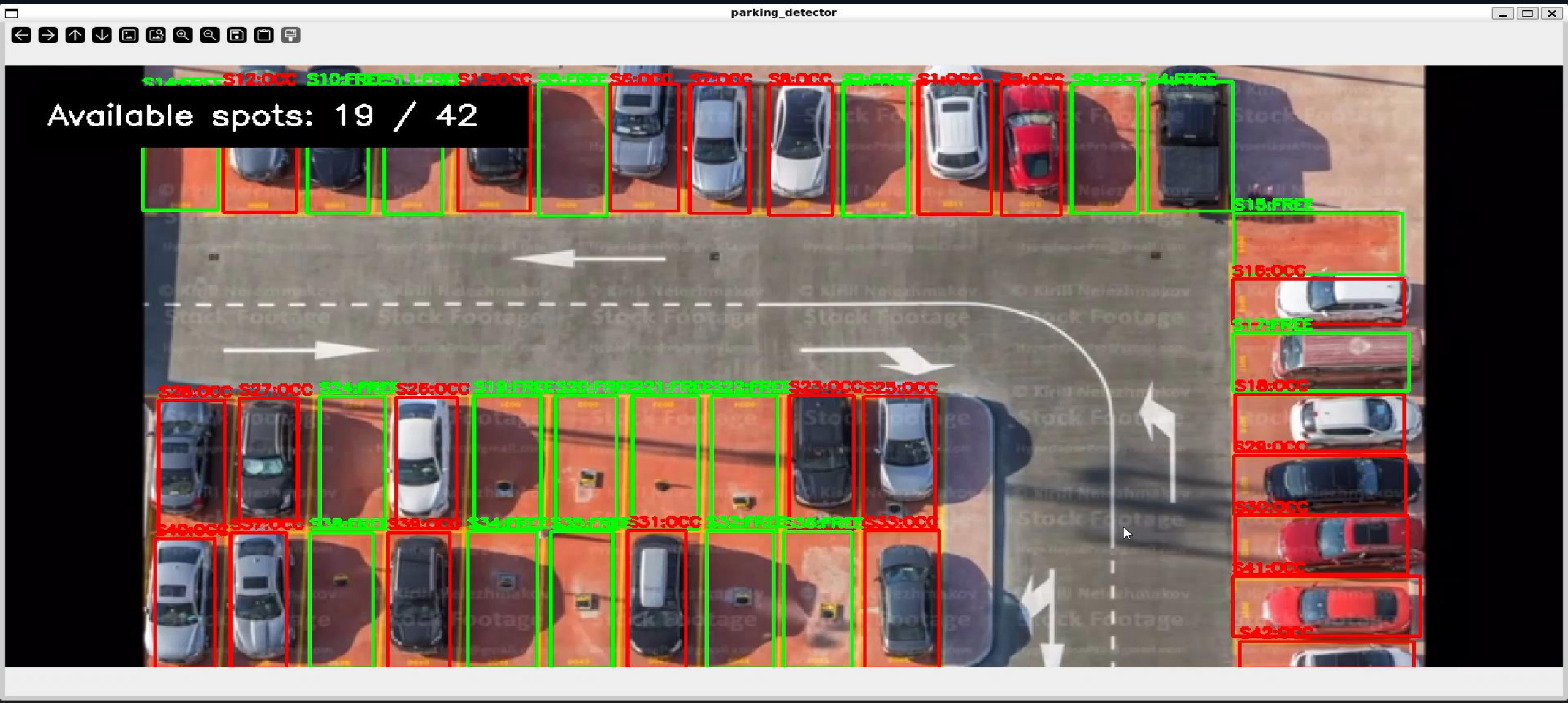
   Use a name field and show the reservation update.



Transition line for the presenter

"Now that you have seen how the pieces connect, I am going to show the prototype working through the dashboard."

Camera Detection Demo



If we had more time

Actual cameras

Connect real campus cameras or live streams instead of relying on static demo assets.

Lot automation

Automatically adapt to each parking lot, camera angle, and side-view layout.

YOLO + SAM

Use object detection and segmentation to identify vehicles and parking boundaries more robustly.

Evaluation + deployment

Measure accuracy, improve edge cases, and deploy the dashboard for real users.

Goal: move from a working prototype to a scalable smart-parking system.

From camera view to useful parking decisions

This project shows a complete path from visual sensing to a working user interface: detect, publish, display, and reserve.

Questions?

ParkingDetectionSoftware

github.com/DawsonHWright/ParkingDetectionSoftware